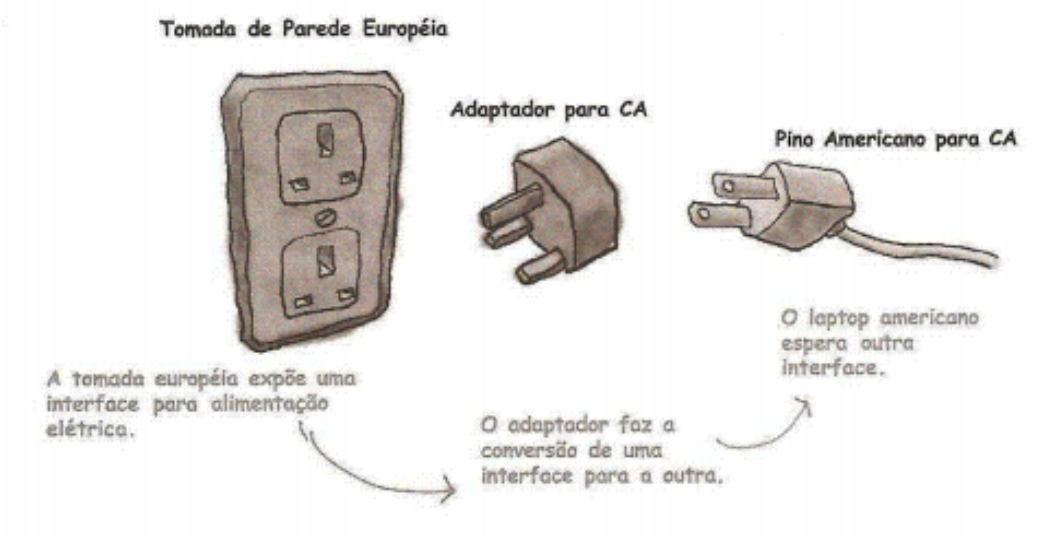
# Métodos Avançados de Programação MAP

## Padrões GoF – Adapter

Sabrina de F. Souto.

# O mundo real está cercado de adaptadores

- Você já tentou usar um laptop fabricado nos EUA em um país europeu?
  - Nesse caso, você provavelmente precisou de um adaptador para ligá-lo à tomada...

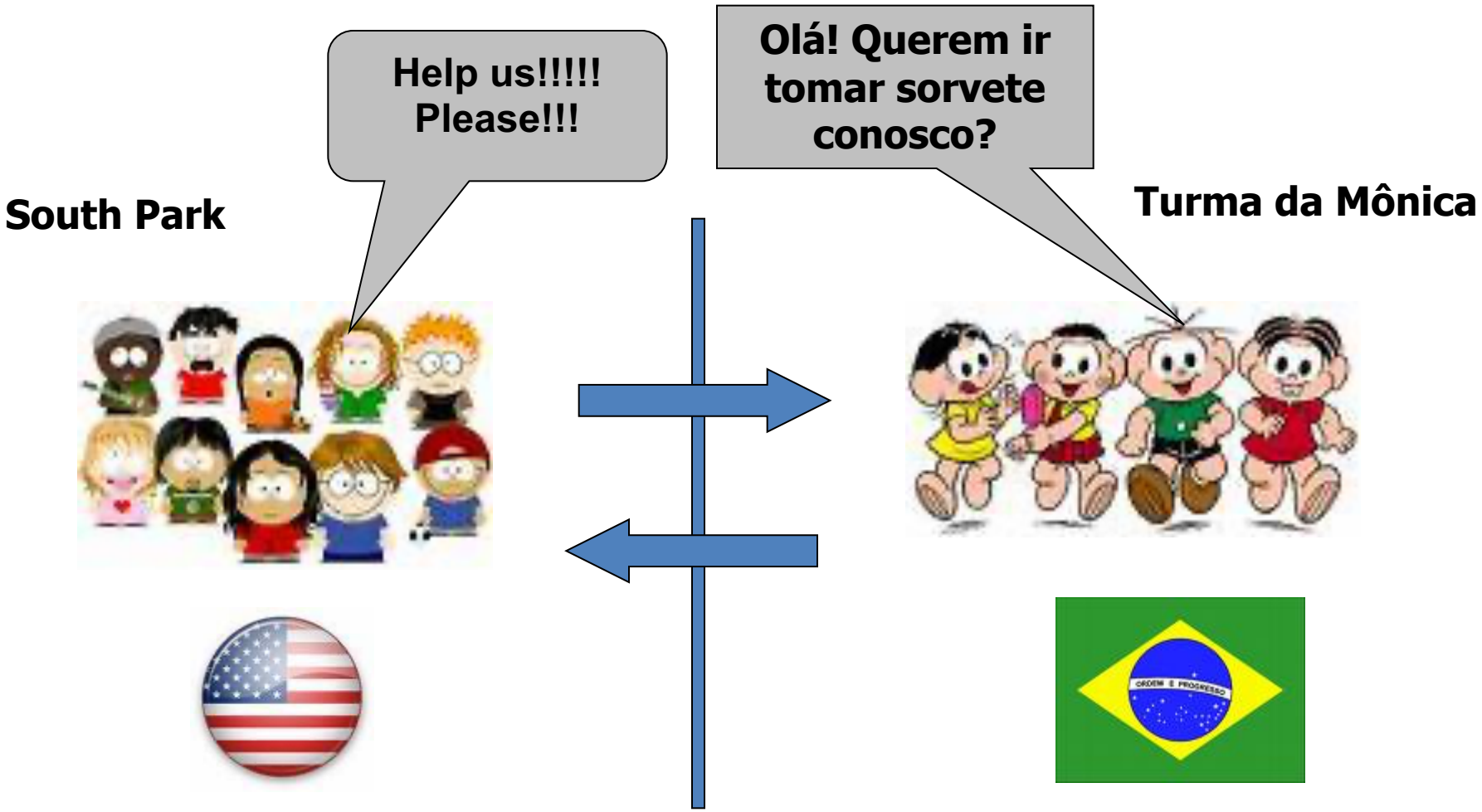



  - Em termos gerais, um adaptador muda a interface da tomada de parede para algo que o seu laptop espera encontrar.

# E os adaptadores OO?

- Desempenham o mesmo papel dos seus equivalentes no mundo real:
  - Eles recebem uma interface e a adaptam para algo que o cliente está esperando.

# O problema


Help us!!!!!
Please!!!
Olá! Querem ir tomar sorvete conosco?
South Park
Turma da Mônica

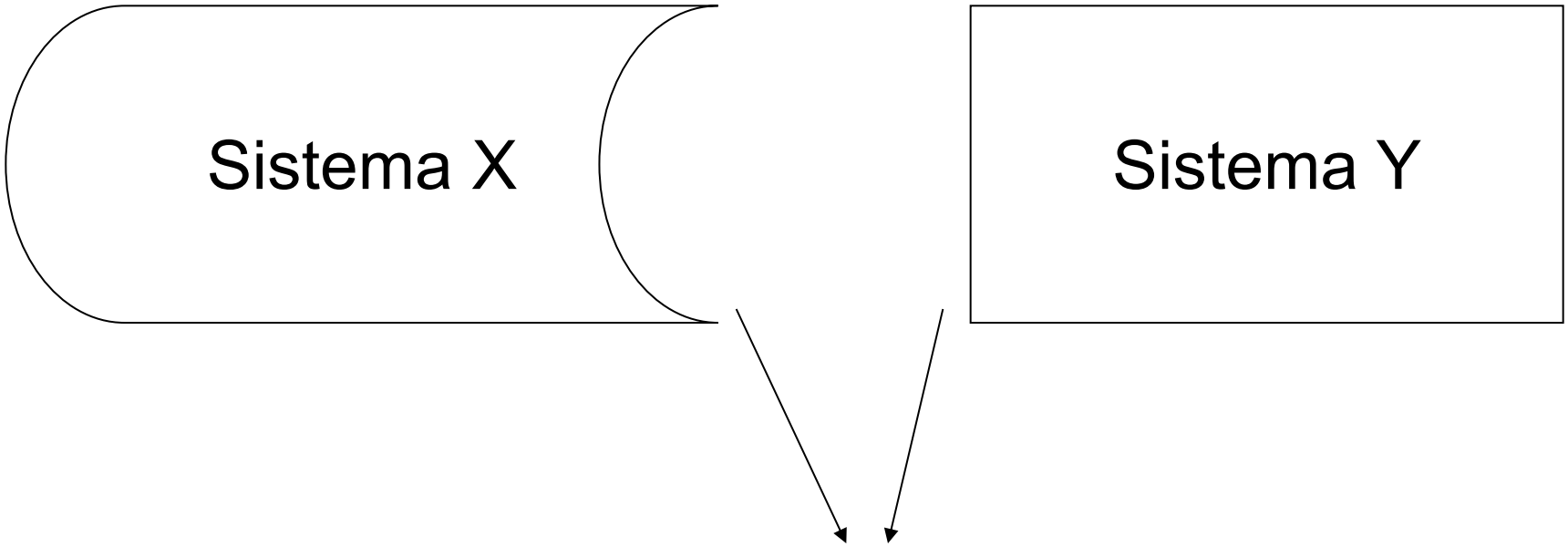
# O problema

- As interfaces de comunicação são distintas
- Não é possível mudar (a curto prazo, por exemplo) uma das interfaces para que se torne compatível à outra
- A comunicação precisa existir.

# O problema

- De maneira mais real
  - Existe um sistema X que funciona
  - Surge a necessidade do sistema X trabalhar com o sistema Y
  - Os sistemas X e Y possuem interfaces distintas (portanto não de integram)

# O problema




Interfaces incompatíveis

# O problema

- Não queremos mudar o nosso sistema (Sistema X), pois este já trabalha com diversos outros com a presente interface.
- Também, não podemos mudar a interface do outro sistema (Sistema Y).
- O que fazer?

Padrão Estrutural

# Adapter

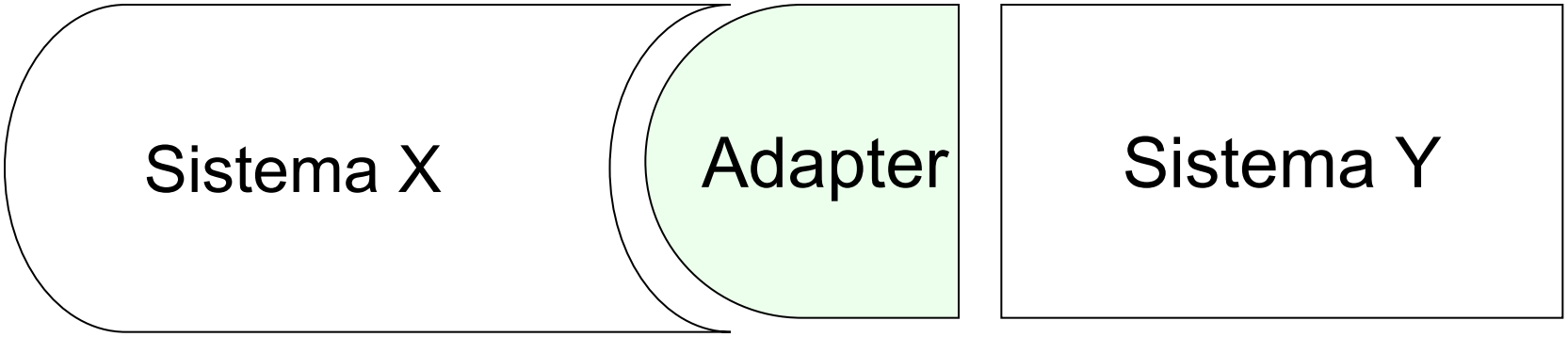
# Objetivo

- Permitir que classes trabalhem conjuntamente apesar de interfaces incompatíveis.
- Como?
  - Convertendo a interface de um sistema em outra interface com a qual os clientes estão prontos para lidar.

# Resumo

- Criar uma classe de adaptação (Adapter) entre as duas interfaces
- O Adapter conhece ambas as interfaces
- Esta classe faz a conversão das requisições (mensagens) entre os dois sistemas

# Resumo



# Quando usar?

- Quando desejamos usar uma classes existente e sua interface não é a desejada ou esperada/apropriada.

# Um exemplo simples

```
public interface Pato {
    public void grasnar ();
    public void voar();
}
```

```
public interface Peru {
    public void gorgolejar ();
    public void voar();
}
```

```
public class PatoReal implements Pato {
        public void grasnar() {
            System.out.println("Quack-Quack");
        }
        public void voar() {
            System.out.println("Estou voando");
        }
}
```

```
public class PeruSelvagem implements Peru {
        public void gorgolejar () {
            System.out.println("Gobble-gobble");
        }
        public void voar() {
            System.out.println("Eu vôo pequenas distâncias");
        }
}
```

# Um peru adaptado

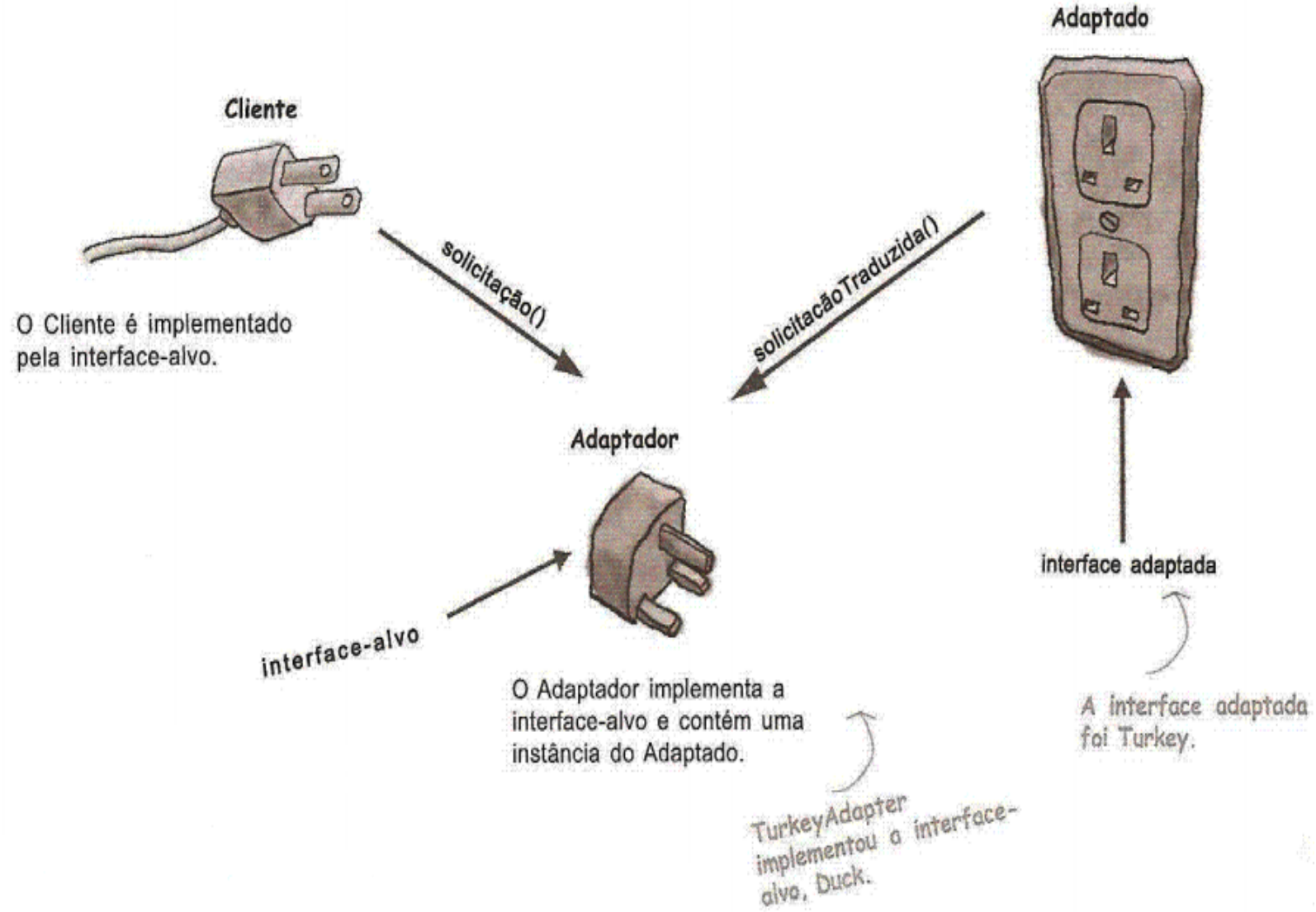
Implementar a interface do tipo que está sendo adaptada.

```
public class PeruAdapter implements Pato {
    private Peru peru;

    public PeruAdapter(Peru peru){
        this.peru = peru;
    }
    public void grasnar () {
            peru. gorgolejar ();
        }

        public void voar() {
            for(int i=0; i<5; i++) {
                peru.voar();
            }
        }
}
```

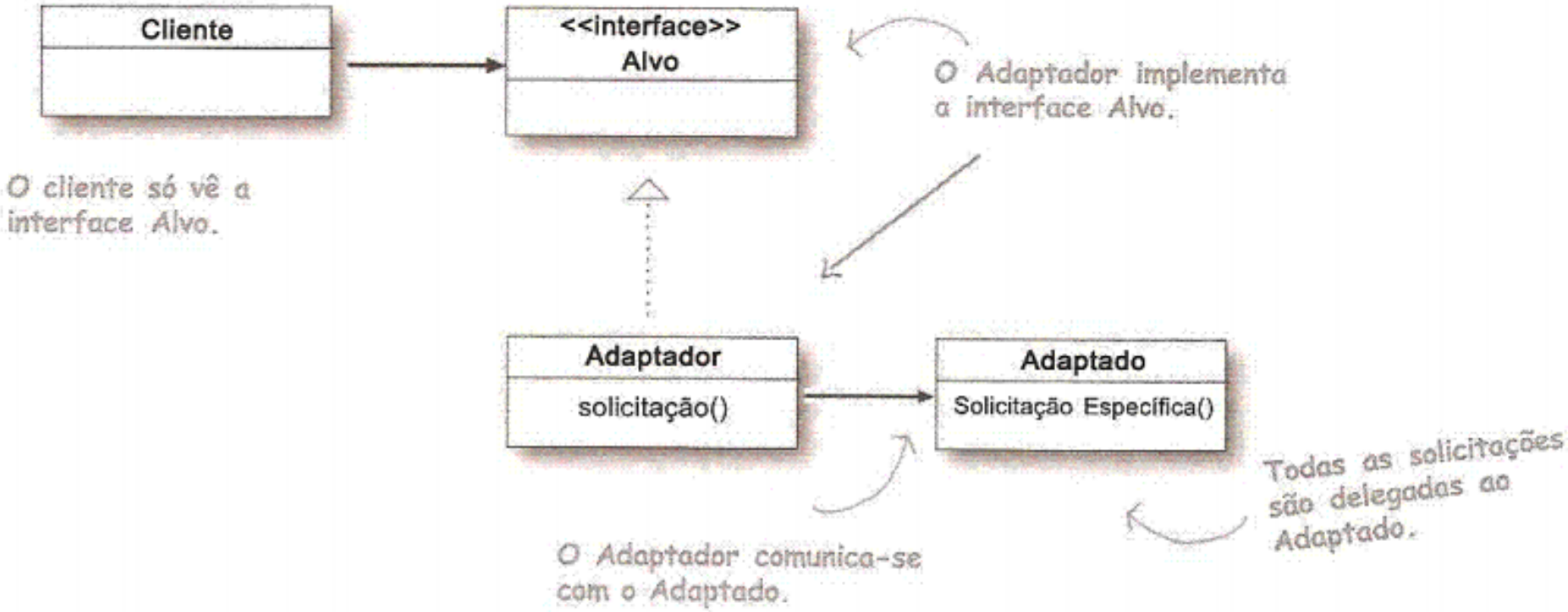
Obter uma referência do objeto que estamos adaptando.

Implementar todos os métodos da interface.

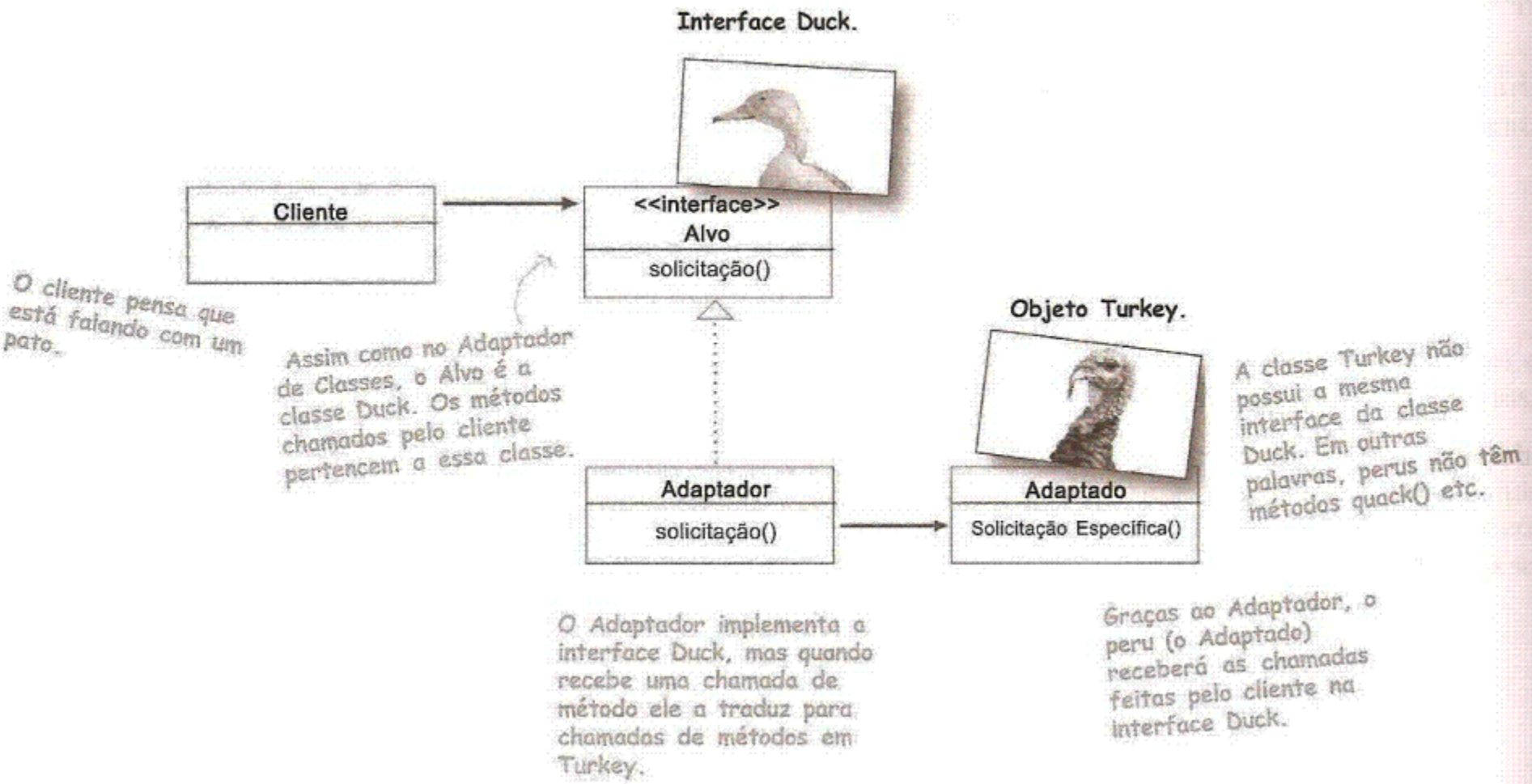
# Explicando o padrão Adapter



# Estrutura Genérica



# Estrutuda do nosso exemplo



# Comentários e consequências

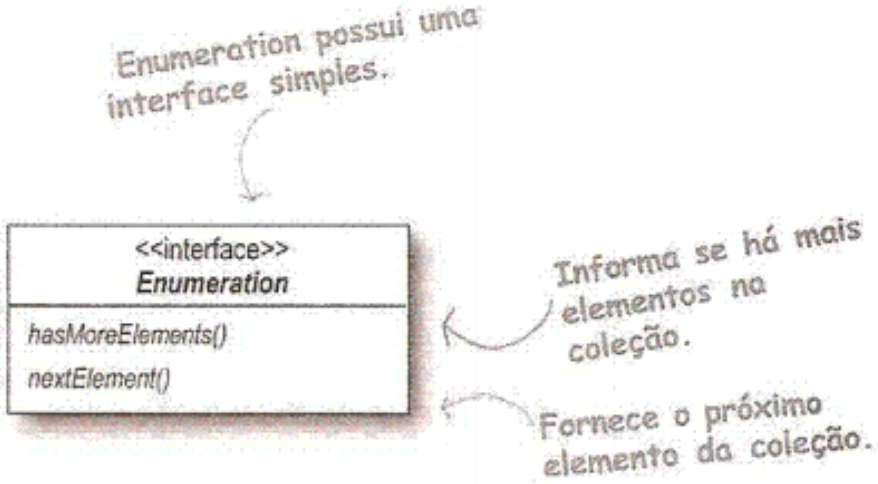
- O cliente conhece uma interface, não uma implementação
- Uso de composição
- O Adaptador pode usar qualquer subclasse de Adaptado

# Um outro exemplo

- A seguir discutiremos outro exemplo um pouco mais real.

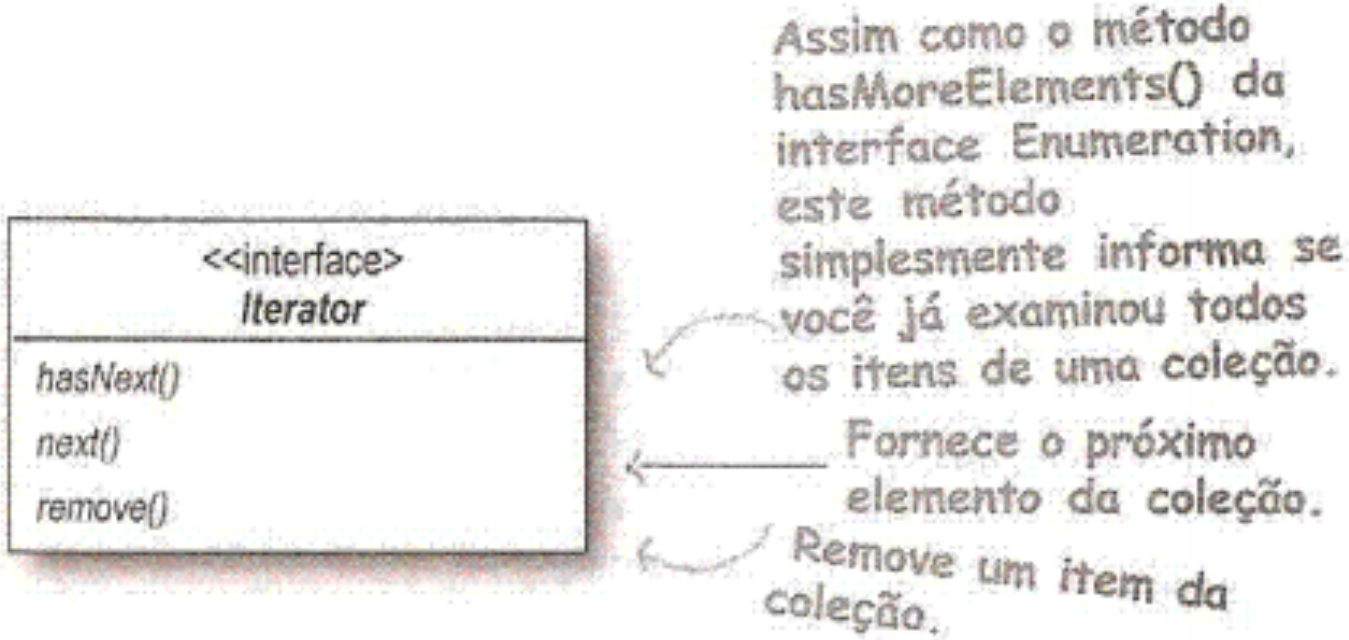
# Enumeration

- Nas primeiras versões Java, as coleções implementavam método elements()
- Este método retornava uma *Enumeration* dos seus elementos.
  - A interface Enumeration permite que percorramos os elementos de uma coleção de forma abstrata (sem saber como estes estão armazenados)



# Iterator

- Nas versões mais recentes de Java, as coleções começaram a utilizar a interface *Iterator.*
  - A interface *Iterator* pode ser vista como uma extensão da interface *Enumeration*, que permite remover elementos.

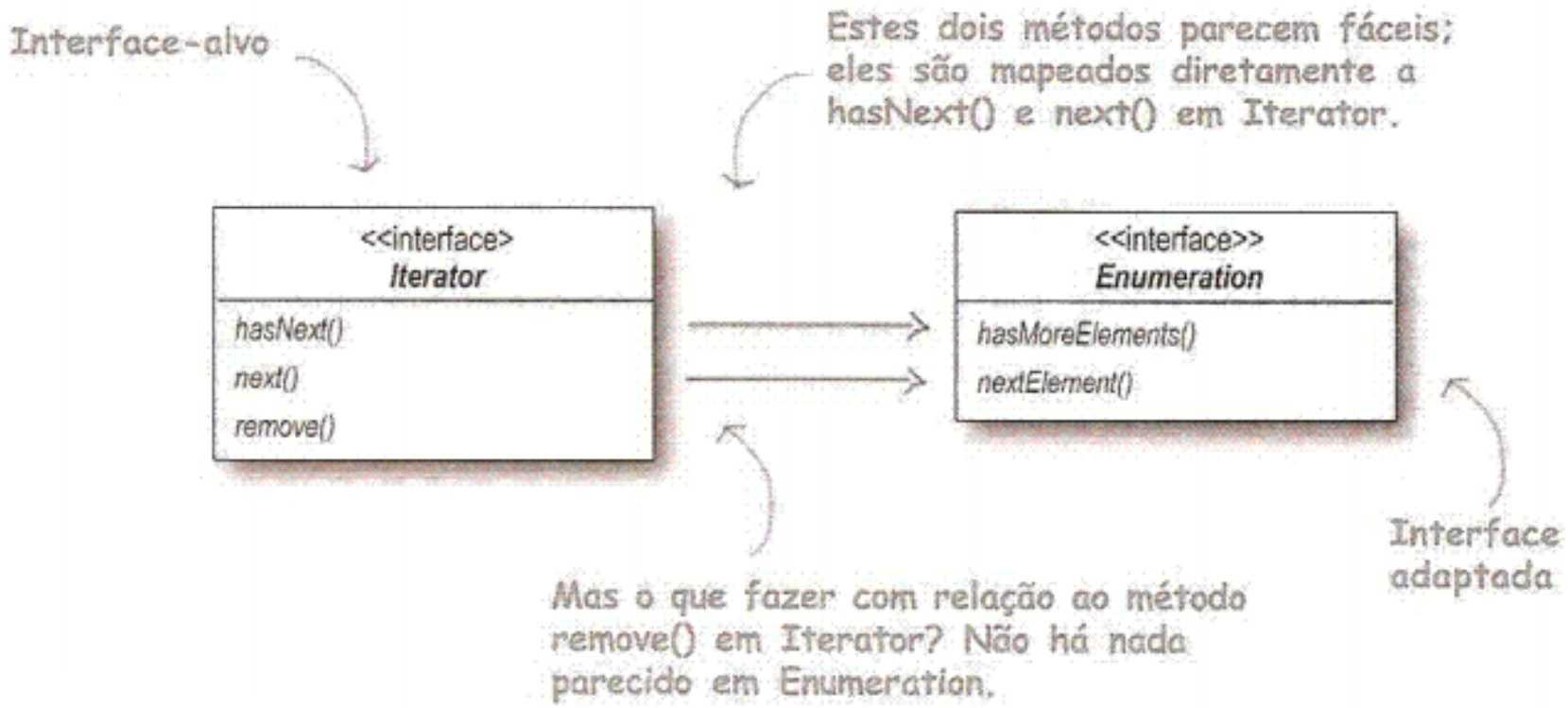


# Enumeration x Iterator

- Diversos sistemas legados ainda estão baseado em *Enumeration*
  - **<u>Caso 1</u>** → Se tivermos um sistema novo: queremos usar *Iterator.*
  - **<u>Caso 2</u>** → Se tivermos um sistema mais antigo: queremos usar Enumeration.
- Suponhamos que não seja possível modificar nenhum dos sistema

# Como resolver?

- Portanto, temos problemas!
- Vamos resolvê-los usando o padrão de projeto *Adapter!!!*



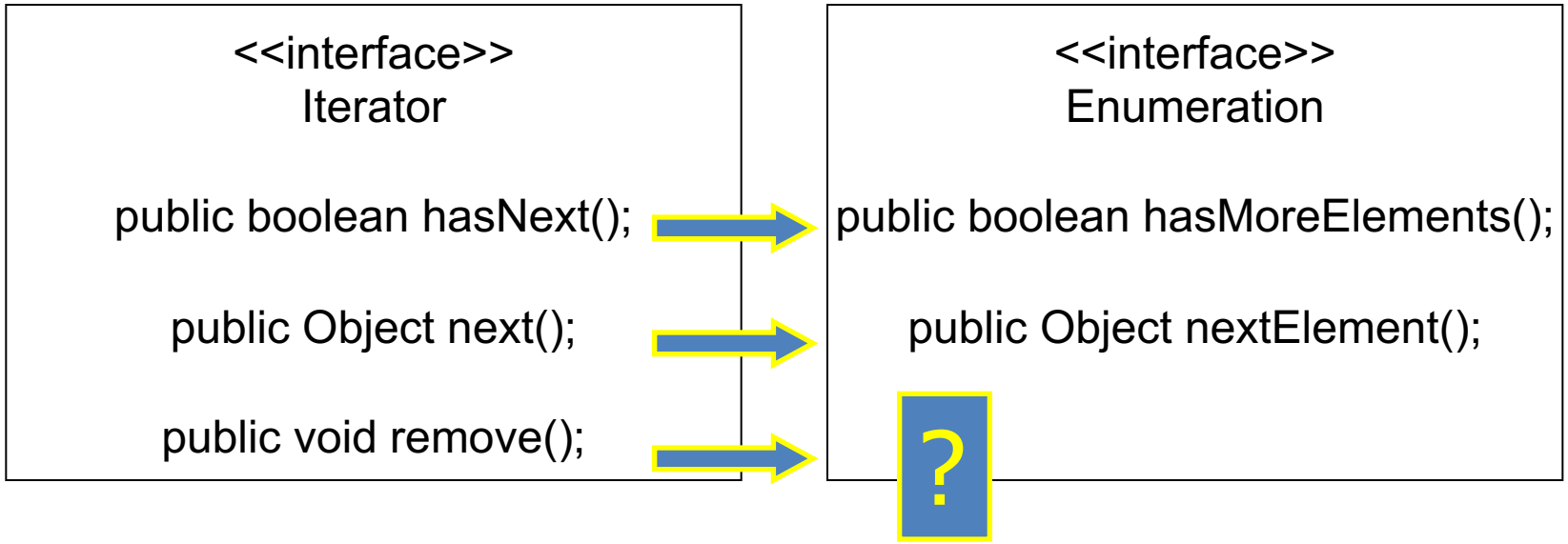
# Alguns comentários

- Note a correlação entre os métodos de ambas interfaces
  - hasMoreElements() ↔ hasNext()
  - nextElement() ↔ next()

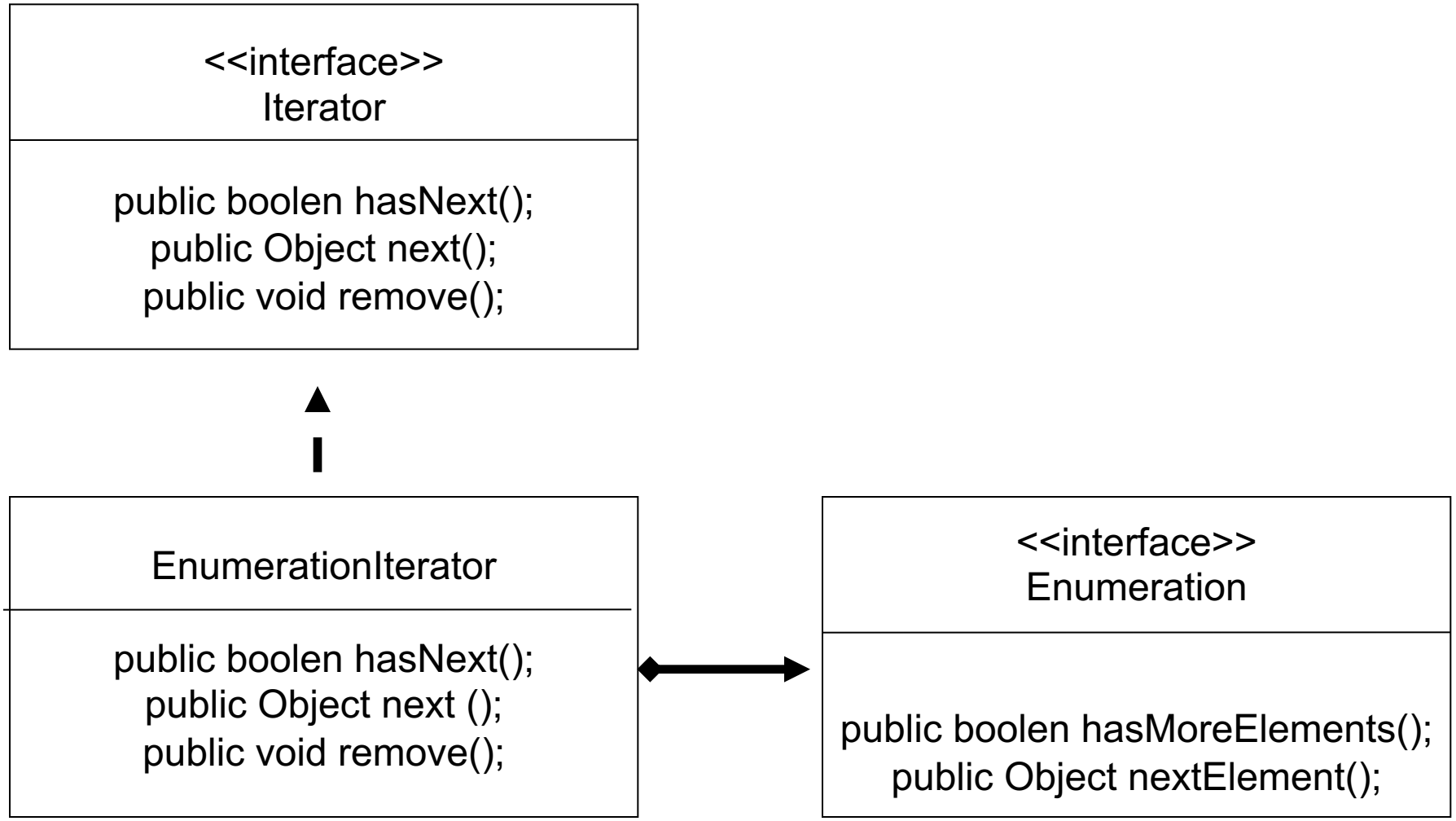
- remove() é uma operação opcional
  - Deve-se lançar ***UnsupportedOperationException*** caso esta operação não seja implementada pelo *Iterator*

# Solução do caso 1: temos um novo sistema

## Queremos usar Iterator



# A solução do caso 1

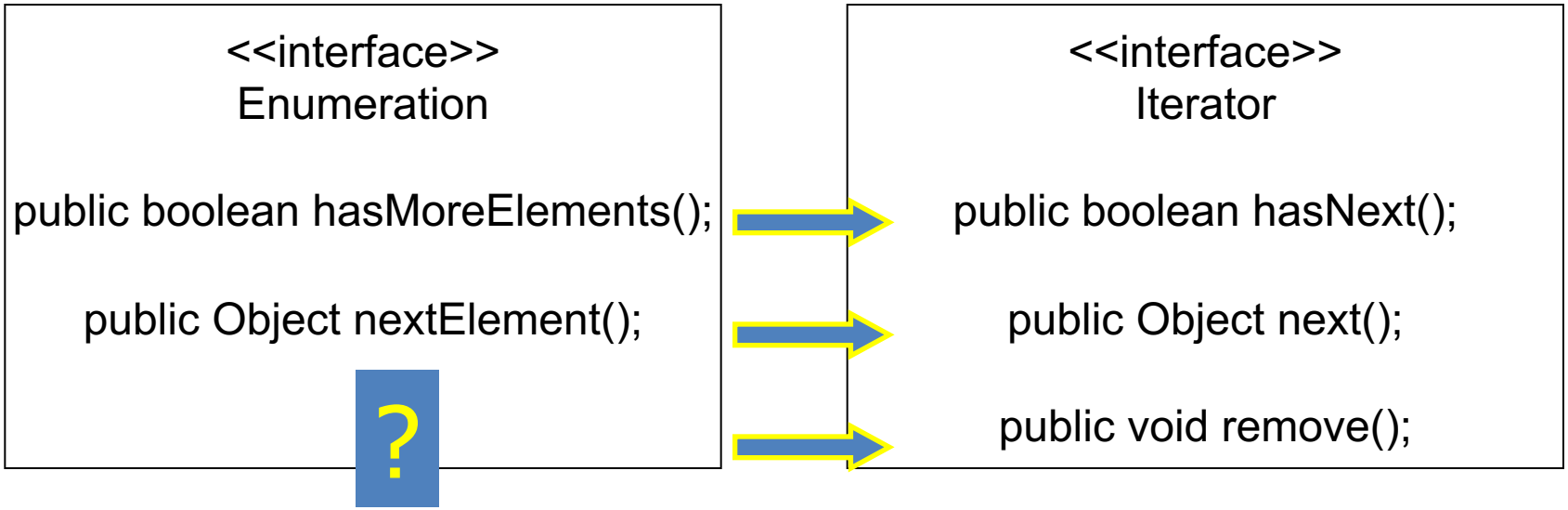


# A solução do caso 1

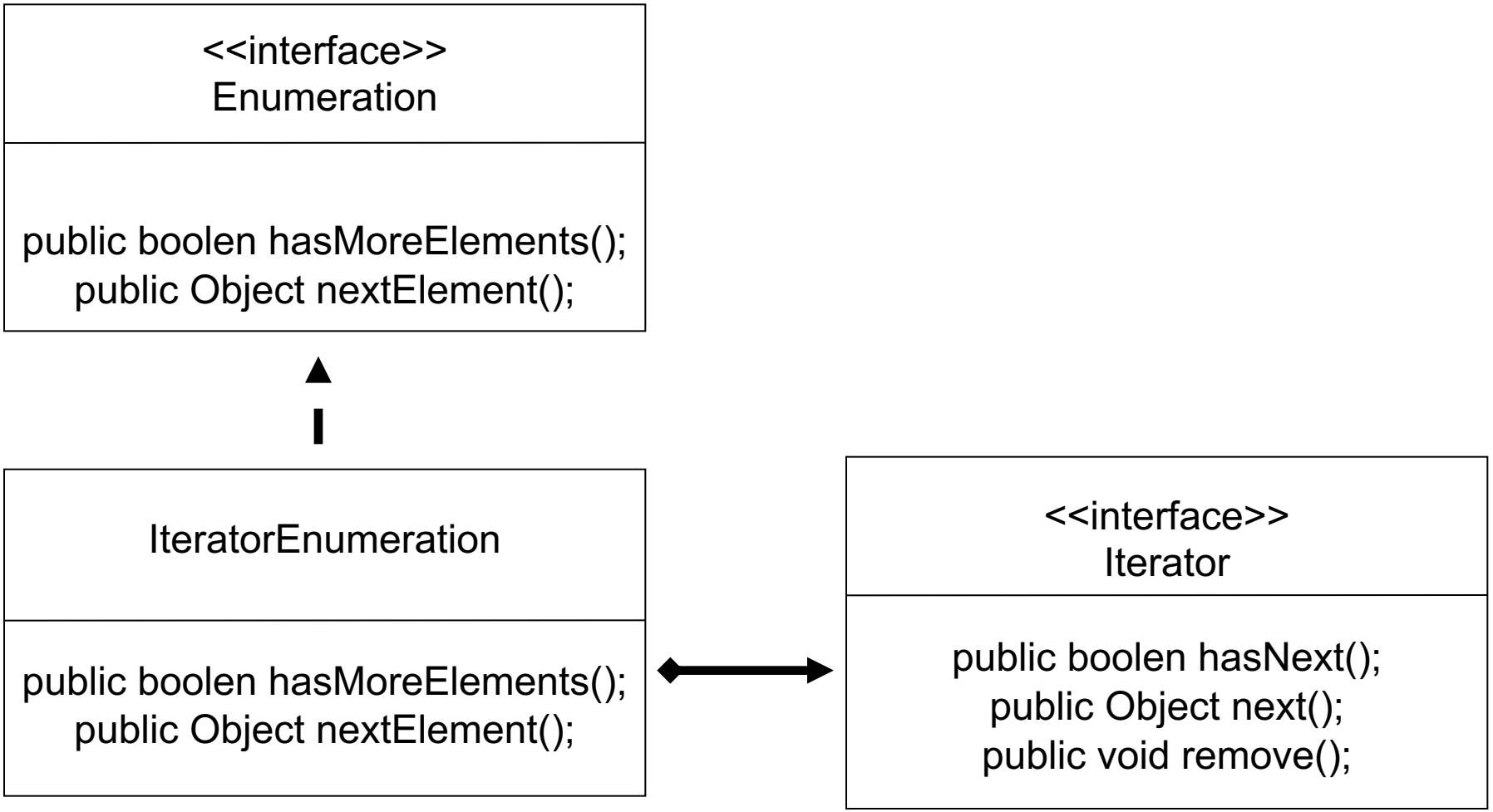
```
public class EnumerationIterator implements Iterator {
  private Enumeration enum;

  public EnumerationIterator(Enumeration enum) {
      this.enum = enum;
  }
  public boolean hasNext() {
      return enum.hasMoreElements();
  }
  public Object next() {
      return enum.nextElement();
  }
  public void remove() {
      throw new UnsupportedOperationException();
  }
}
```

# Solução do caso 2: temos um Sistema legado

## Queremos usar Enumeration



# A solução do caso 2



# A solução do caso 2

```
public class IteratorEnumeration implements Enumeration {
    private Iterator it;

    public IteratorEnumeration(Iterator it){
     this.it = it;
    }

    public boolean hasMoreElements () {
     return it.hasNext();
    }

    public Object nextElement() {
     return it.next ();
    }
}
```

# Dúvida?